

فاز ۲: برنامه‌نویسی شی‌گرا (Object-Oriented)

(Programming) – کامل و عمیق

برنامه‌نویسی شی‌گرا فقط یاد گرفتن ۴ تا کلمه نیست (کپسوله‌سازی، وراثت، چندریختی، انتزاع). باید بفهمی این مفاهیم چه مشکلی رو حل می‌کنن، چطور توی کد واقعی پیاده میشن، و چرا اینقدر مهم هستن.

2.0 چرا اصلاً شی‌گرایی؟ مشکل برنامه‌نویسی رویه‌ای (Procedural) چی بود؟

تصور کن یه برنامه بانکی داری. هزاران تابع داری: `calculateInterest()`, `deposit()`, `withdraw()`... همه این توابع روی داده‌های سراسری (Global Variables) کار می‌کنن.

• مشکل ۱: هر تابعی می‌تونه داده رو خراب کنه. هیچ محافظی نیست.

• مشکل ۲: اگه ساختار داده عوض بشه (مثلاً حساب پس‌انداز بشه دو نوع: کوتاه‌مدت و بلندمدت)، باید تمام توابع رو عوض کنی. این همیشه یه فاجعه.

• مشکل ۳: کد برای تیم‌های بزرگ و پروژه‌های عظیم غیرقابل مدیریت میشه.

شی‌گرایی این راه حل رو میده: داده و توابعی که روی اون داده کار می‌کنن رو بذار توی یه بسته به اسم شیء (Object) و بگو هیچکی حق نداره مستقیماً به داده دست بزنه. فقط از طریق اون توابع می‌تونی.

2.1 کلاس و شیء (Class & Object)

• کلاس: یه قالب یا نقشه هست. هیچ حافظه‌ای اشغال نمی‌کنه. (مثل نقشه ساختمان یا فرم خالی ثبت‌نام).

• شیء: یه نمونه واقعی از روی اون قالب که حافظه اشغال می‌کنه. (مثل ساختمونی که ساخته شده یا فرم پُر شده).

! سوال مصاحبه‌ای ۱: توی حافظه، وقتی یه شیء می‌سازی دقیقاً چه اتفاقی می‌افته؟

جواب:

1. وقتی JVM به دستور new میرسه، میره توی Heap فضای کافی برای فیلدهای اون کلاس (و کلاس‌های والدی) کنار میذاره.

2. اون فضا رو با مقادیر پیش‌فرض (0, null, false) پر می‌کنه.

3. Constructor (سازنده) کلاس رو صدا میزنه تا فیلدها رو با مقادیر دلخواه ما مقداردهی اولیه کنه.

4. در نهایت، آدرس این شیء توی Heap، به یه متغیر Reference در Stack داده میشه.

1 // *a. توی اشاره‌گر توی Stack اینجا فقط یه اشاره‌گر توی*

2 `BankAccount myAccount;`

3

4 // *b. ذخیره میشه myAccount اینجا اتفاقی بالا میفته و آدرس شیء جدید توی*

5 `myAccount = new BankAccount();`

2.2 سازنده‌ها (Constructors)

سوال: Constructor چیه و فرقی با متد معمولی چیه؟

جواب:

• هیچ نوع بازگشتی‌ای نداره (حتی void).

• همنام کلاسه.

• برای مقداردهی اولیه به فیلدهای شیء هنگام ساختش استفاده میشه.

• اگه هیچ سازنده‌ای نسازی، جاوا یه سازنده پیش‌فرض (Default Constructor) بدون پارامتر برات میسازه. ولی به محض این که خودت یه سازنده (حتی با پارامتر) بسازی، اون پیش‌فرض دیگه ساخته نمیشه.

.

```

1 public class Person {
2     private String name;
3
4     // سازنده (Constructor)
5     public Person(String name) {
6         this.name = name; // this.name اشاره میکنه
7     }
8 }
9 // حالا این غلطه: Person p = new Person();
10 // این درسته: Person p = new Person("Ali");

```

کلمه کلیدی **this**: همیشه به شیء فعلی (همینی که باهاش کار می‌کنیم) اشاره داره. برای فرق گذاشتن بین فیلد کلاس و پارامتر ورودی استفاده میشه

2.3 اصل اول: کپسوله‌سازی (Encapsulation)

تعریف ساده: بسته‌بندی داده‌ها (متغیرها) و متدهایی که روی آن‌ها کار می‌کنند، در یک واحد (کلاس)، و مخفی کردن جزئیات پیاده‌سازی از دنیای بیرون.

چرا مهمه؟

• امنیت: نمی‌داری بقیه مستقیم بیان و فیلدهای کلاس رو خراب کنن.

• انعطاف‌پذیری: هر وقت خواستی می‌تونی منطق داخلی رو عوض کنی. تا وقتی امضای متدهای **public** عوض نشه، هیچ کد دیگه‌ای در برنامه نمی‌شکنه.

چطور پیاده‌سازی میشه؟ با Access Modifiers (سطوح دسترسی):

Modifier	داخل کلاس	داخل Package	زیرکلاس (ارث‌بری)	همه جا
private	✓	✗	✗	✗
default (بدون کلمه)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

قانون طلایی برنامه‌نویسان حرفه‌ای

• فیلدها رو همیشه private کن.

• دسترسی بهشون رو فقط از طریق متدهای public (getter و setter) بده. به اینا POJO یا Java Bean هم میگن.

```

1 public class Employee {
2     // ۱. مخفی‌سازی داده
3     private double salary;
4
5     // ۲. دسترسی کنترل‌شده (Getter)
6     public double getSalary() {
7         // اجازه دیدن بدی HR اینجا می‌تونی منطق بذاری. مثلاً فقط به
8         return salary;
9     }
10
11    // ۳. تغییر کنترل‌شده (Setter)
12    public void setSalary(double newSalary) {
13        if (newSalary > 0) { // اعتبارسنجی
14            this.salary = newSalary;
15        }
16    }

```

2.4 اصل دوم: وراثت (Inheritance)

تعریف ساده: ساختن کلاس جدید بر اساس کلاس موجود. کلاس جدید (زیرکلاس/فرزند) فیلدها و متدهای کلاس موجود (آبرکلاس/والد) را به ارث می‌برد. رابطه IS-A داریم: Dog IS-A Animal.

مزیت اصلی: استفاده مجدد از کد (Code Reusability).

! سوال مصاحبه‌ای: تفاوت Association،

Aggregation و Composition رو کامل توضیح بده

جواب: این سه تا رابطه، درجه‌های مختلف وابستگی بین دو کلاس رو نشون میدن. از ضعیف به قوی:

۱. Association (ارتباط) – ضعیف‌ترین

تعریف: وقتی دو کلاس با هم ارتباط دارن، اما هیچکدوم مالک دیگری نیست و هیچ وابستگی عمری (Lifecycle Dependency) بینشون نیست.

تشبیه: من و یه دکتر. من می‌رم پیش دکتر، ولی دکتر مال من نیست و من مال دکتر نیستم. هرکدوم می‌تونیم بدون دیگری وجود داشته باشیم.

علامت: Uses a / Is associated with

2-Java-OOP

```
1 // Association - کاملاً مستقل
2 class Doctor {
3     public void treat(Patient patient) {
4         // دکتر بیمار رو درمان می‌کنه
5     }
6 }
7
8
9 // استفاده
10 Doctor doc = new Doctor();
11 Patient p = new Patient();
12 doc.treat(p); // ارتباط، ولی مالکیت نیست
```

ویژگی‌ها:

- هر دو شیء مستقل هستند
- هیچکدام مالک دیگری نیست
- عمر هرکدام به دیگری وابسته نیست
- می‌تونه یک‌طرفه یا دوطرفه باشه

۲. Aggregation (تجميع) – متوسط

تعريف: رابطه HAS-A ضعيف. يعنى يه كلاس شامل كلاس ديگه ست، ولى اون كلاس مى تونه مستقل از مالکش هم وجود داشته باشه.

تشبيه: دانشگاه و دانشجوها. دانشگاه دانشجوها رو داره، ولى اگه دانشگاه تعطيل بشه، دانشجوها همچنان وجود دارن. (قبلاً گفتيم)

علامت: Has a (weak)

```

1 // Aggregation - جزء مى تونه مستقل وجود داشته باشه
2 class Student {
3     private String name;
4     // Student وجود داشته باشه University مى تونه بدون
5 }
6
7 class University {
8     private List<Student> students;
9
10    public University(List<Student> students) {
11        this.students = students; // از بیرون داده میشه
12        // ها هنوز هستن Student، از بین بره University اگه
13    }
14 }
15
16 // استفاده
17 Student s1 = new Student();
18 Student s2 = new Student();

```

```
1 List<Student> list = Arrays.asList(s1, s2);
```

```
2 University uni = new University(list); // دانشجویها قبل از دانشگاه بودن
```

ویژگی‌ها:

• شیء جزء می‌تونه بدون شیء کل وجود داشته باشه

• در Constructor از بیرون دریافت میشه (نه اینکه داخل ساخته بشه)

• رابطه "کل-جزء" (Whole-Part) هست

۳. Composition (ترکیب) – قوی‌ترین

تعریف: رابطه HAS-A قوی. یعنی جزء، بخشی از کل هست و بدون کل معنی نداره. اگه کل از بین بره، جزء هم از بین می‌ره.

تشبیه: خانه و اتاق‌ها. اتاق‌ها بدون خانه معنی ندارن. اگه خانه خراب بشه، اتاق‌ها هم از بین میرن.

علامت: Has a (strong) / Part of

```
1 // Composition - جزء بدون کل وجود نداره
2 class Room {
3     // Room معنی نداره House بدون Room
4 }
5
6 class House {
7     private Room room; // Composition
8
9     public House() {
10         this.room = new Room(); // داخل خودش ساخته میشه
```



```

1  // هم می‌میره Room، از بین بره House اگه
2  }
3 }
4
5 // استفاده
6 House house = new House(); // هم ساخته میشه House، Room با ساخت
7 // هم از بین می‌ره room، از بین بره house وقتی

```

ویژگی‌ها:

- شیء جزء بدون شیء کل وجود نداره
- داخل Constructor خود کل ساخته میشه (نه از بیرون)
- با نابودی کل، جزء هم نابود میشه

جدول مقایسه سه رابطه

ویژگی	Association	Aggregation	Composition
قدرت	ضعیف	متوسط	قوی
وابستگی عمری	هیچی	کم	کامل
جزء مستقل؟	کاملاً مستقل	مستقل	وابسته به کل
مثال	دکتر و بیمار	دانشگاه و دانشجو	خانه و اتاق
در کد	پارامتر متد	از بیرون دریافت	داخل ساخته میشه
نماد UML	خط ساده 	لوزی توخالی — — —	لوزی پر — — —

🎯 خلاصه کاربردی برای مصاحبه

- 1 Association (ارتباط) → "Uses a" → ضعیف‌ترین
- 2 Aggregation (تجمیع) → "Has a" → متوسط (جزء مستقل)
- 3 Composition (ترکیب) → "Has a" → قوی‌ترین (جزء وابسته)

نکته طلایی برای مصاحبه:

"همیشه سعی کن به جای Inheritance (IS-A) از Composition (HAS-A) استفاده کنی. این قانون طلایی طراحی شی‌گراست."

مثال کامل در یک کلاس

```

1 // ۱. Association - فقط استفاده
2 class Driver {
3     public void drive(Car car) { // Association
4         car.move();
5     }
6 }
7
8 // ۲. Aggregation - جمع شدن
9 class Player {
10     private String name;
11     // Player می‌تونه تنها هم باشه
12 }
13

```

```

1 class Team {
2     private List<Player> players; // Aggregation
3
4     public Team(List<Player> players) {
5         this.players = players; // از بیرون میاد
6     }
7 }
8
9 // ۳. Composition - بخشی از کل
10 class Engine {
11     // Engine نداره Car بدون Engine
12 }
13
14 class Car {
15     private Engine engine; // Composition
16
17     public Car() {
18         this.engine = new Engine(); // داخل ساخته میشه
19     }
20 }

```

! سوال مصاحبه‌ای ۲: چرا "ترکیب (Composition) را بر وراثت ترجیح بده" به اصل طراحی

معروفه؟ مشکل وراثت چیه؟

جواب حیاتی: وراثت باعث ایجاد جفت‌شدگی شدید (Tight Coupling) بین والد و فرزند میشه. یعنی:

• تغییر در کلاس والد می‌تونه تمام کلاس‌های فرزند رو بشکند. (بهبش میگن مشکل Fragile Base Class).

• وراثت، جزئیات پیاده‌سازی والد رو برای فرزند فاش می‌کنه (نقض کپسوله‌سازی).

• در جاوا که فقط یه والد می‌تونی داشته باشی، خیلی محدودکننده است.

ترکیب (Composition): یعنی به جای extends کردن، یه نمونه از اون کلاس رو به عنوان یه فیلد در کلاس خودت داشته باشی (HAS-A). اینطوری انعطاف‌پذیری خیلی بالاتری داری و می‌تونی رفتار رو در زمان اجرا عوض کنی.

```

1 // a. (وراثت) روش بد:
2 class TaxCalculator {
3     double calculate() { ... }
4 }
5 class IranTaxCalculator extends TaxCalculator { ... } // tight coupling
6
7 // b. (ترکیب) روش خوب:
8 interface TaxStrategy { double calculate(); }
9 class IranTax implements TaxStrategy { ... }
10 class USATax implements TaxStrategy { ... }
11
12 class Invoice {
13     private TaxStrategy tax; // Composition!
14
15     public Invoice(TaxStrategy tax) {
16         this.tax = tax;
17     }
18 }
19 // c. حالا هر وقت خواستی، استراتژی مالیاتی رو عوض می‌کنی
20 new Invoice(new IranTax());

```

```
21 new Invoice(new USATax());
```

! سوال مصاحبه‌ای ۲.۵: فرق Composition و Aggregation چیه؟

جواب: هر دو نوعی رابطه HAS-A هستند، ولی تفاوت در طول عمر هست:

• Composition (ترکیب - قوی): شیء فرزند بدون والد وجود نداره. اگر والد بمیره، فرزند هم می‌میره.

• Aggregation (تجمع - ضعیف): شیء فرزند می‌تونه مستقل از والد وجود داشته باشه.

```
1 // Composition - خانه و اتاق‌ها
2 class Room { ... }
3 class House {
4     private Room room; // با ساخت خانه، اتاق ساخته میشه
5     public House() {
6         this.room = new Room(); // اتاق فقط درون خانه معنی داره
7     }
8 }
9
10 // Aggregation - دانشگاه و دانشجو
11 class Student { ... }
12 class University {
13     private List<Student> students; // دانشجو قبل از دانشگاه وجود داشته
14     public University(List<Student> students) {
15         this.students = students; // دانشجو مستقل از دانشگاه هست
16     }
17 }
18
19 ## 2.5 چندریختی (Polymorphism)
```

1

2 از نوع والد، برای اشاره به اشیاء از نوع فرزند. یا صدا زدن یه (Reference) تعریف ساده: **"چندشکلی"**. توانایی یه **ارجاع** ****** متد یکسان و گرفتن رفتارهای متفاوت بر اساس نوع شیء

3

4 دو نوع داریم:

5

6 Method Overloading یا (Compile-Time) الف چندریختی زمان **کامپایل** **###**

7

8 یه کلاس، چندین متد با نام یکسان ولی پارامترهای **متفاوت** (تعداد، نوع، یا ترتیب) داره. کامپایلر با توجه به آرگومان‌های ورودی، تشخیص میده کدوم متد صدا زده بشه.

9

10 ```java

11 class Printer {

12 void print(String s) { ... }

13 void print(int i) { ... }

14 void print(String s, int copies) { ... }

15 }

! سوال مصاحبه‌ای ۳: آیا با تغییر نوع بازگشتی می‌تونیم Overload کنیم؟

جواب: نه! نوع بازگشتی امضای متد نیست و به تنهایی برای Overloading کافی نیست. کامپایلر موقع صدا زدن متد، فقط به اسم متد و پارامترها نگاه می‌کنه.

ب) چندریختی زمان اجرا (Runtime) یا Method Overriding

کلاس فرزند، به متد از کلاس والد رو با دقیقاً همون امضا (اسم، پارامترها، نوع بازگشتی) بازنویسی (@Override) می‌کنه. تصمیم‌گیری برای اینکه کدوم نسخه از متد اجرا بشه، در زمان اجرا و بر اساس نوع واقعی شیء گرفته میشه. به این میگن Dynamic Method Dispatch.

```

1 class Animal {
2     void sound() { System.out.println("Some sound"); }
3 }
4 class Dog extends Animal {
5     @Override void sound() { System.out.println("Woof!"); }
6 }
7 class Cat extends Animal {
8     @Override void sound() { System.out.println("Meow"); }
9 }
10
11 public class Main {
12     public static void main(String[] args) {
13         Animal myPet; // به ارجاع از نوع Animal
14
15         myPet = new Dog();
16         myPet.sound(); // رفتار Dog: Woof! خروجی
17
18         myPet = new Cat();
19         myPet.sound(); // رفتار Cat: Meow خروجی
20     }
21 }

```

! سوال مصاحبه‌ای ۴: @Override چیه و آیا اجباریه؟

جواب: اجباری نیست، ولی به شدت توصیه میشه. این Annotation به کامپایلر میگه "من قصد دارم یه متد از والد رو بازنویسی کنم". اگه اشتباهی اسم متد رو غلط بنویسی یا پارامترهایش رو اشتباه بگیری، کامپایلر بهت خطا میده و نمی‌ذاره. بدون @Override، متد یه متد جدید محسوب میشه و چندریختی اتفاق نمی‌افته.

2.6 اصل چهارم: انتزاع (Abstraction)

تعریف ساده: پنهان کردن پیچیدگی و نمایش فقط اطلاعات ضروری. یعنی بگی چه کاری انجام میشه، نه چطور.

چرا مهمه؟ پیچیدگی سیستم رو کم میکنه و به کاربر (یا کلاس دیگه) اجازه میده با یه رابط ساده کار کنه.

چطور پیاده میشه؟

• کلاس انتزاعی (Abstract Class): کلاسیه که نمی‌تونی ازش شیء بسازی (new کنی). فقط برای اینه که

زیرکلاس‌ها ازش ارث‌بری کنن. می‌تونه متدهای abstract (بدون بدنه) و متدهای معمولی داشته باشه.

• رابط (Interface): یه "قرارداد" کامله. قبل از Java 8 فقط می‌تونست متدهای abstract و ثابت‌ها (static final)

داشته باشه. حالا می‌تونه متدهای default و static هم داشته باشه.

! سوال مصاحبه‌ای ۵ (حیاتی): تفاوت Abstract Class و Interface رو کامل و با مثال

توضیح بده

جواب:

کلاس انتزاعی (Abstract Class):

• با extends استفاده میشه (فقط یکی).

• می‌تونه constructor داشته باشه (برای مقداره‌ی به فیلدهای مشترک والد).

• می‌تونه متدهای معمولی (با بدنه) داشته باشه.

• فیلدهاش می‌تونن private, protected و غیره باشن.

• برای وقتی خوبه که کلاس‌های مرتبط، یه پیاده‌سازی مشترک دارن. (مثلاً HttpServlet که کلی کد مشترک داره).

رابط (Interface):

• با implements استفاده میشه (چندین تا می‌تونن پیاده‌سازی کنن).

• constructor نداره.

• متدهاش ذاتاً public abstract بودن. متدهای default و static هم الان داریم.

• فیلدهاش فقط public static final هستن (یعنی ثابت).

• برای وقتی خوبه که می‌خوای یه "قابلیت" یا "رفتار" مشترک بین کلاس‌های کاملاً نامرتبط تعریف کنی

(مثلاً Serializable, Comparable).

```

1 // قرارداد "چه کاری"
2 interface Payable {
3     void pay(int amount); // همه‌شون باید بلد باشن پرداخت کنن
4 }
5
6 // Abstract Class = پایه مشترک
7 abstract class Employee {
8     protected String name;
9     public Employee(String name) { this.name = name; }
10    abstract double calculateSalary(); // هر کارمندی حقوقش فرق داره
11    public void clockIn() { System.out.println(name + " clocked in."); } // کار مشترک
12 }
13
14 // حالا استفاده

```

```

1 class FullTimeEmployee extends Employee implements Payable {
2     public FullTimeEmployee(String name) { super(name); }
3     @Override double calculateSalary() { return 5000; }
4     @Override public void pay(int amount) { System.out.println("Paid via bank"); }
5 }

```

2.7 مفاهیم تکمیلی و حیاتی OOP

! سوال مصاحبه‌ای ۶: static چیست؟ متد static با متد غیر static چه فرقی دارد؟

جواب: static یعنی متعلق به کلاس، نه به نمونه (شیء) خاص.

• فیلد static: به متغیر که بین همه اشیاء اون کلاس مشترک. تغییرش در یکی، در همه اعمال میشه. (مثل شمارنده تعداد اشیاء ساخته شده).

• متد static: می‌تونن بدون ساختن شیء صداش بزنی (ClassName.method()). به فیلدهای غیر static (فیلد شیء) دسترسی نداره، چون اصلاً شیئی وجود نداره که بخواد بهش دسترسی داشته باشه! main هم static هست چون JVM بدون ساختن هیچ شیئی باید بتونه اجراش کنه.

! سوال مصاحبه‌ای ۷: final چیست و کجاها استفاده میشه؟

جواب:

• متغیر final: به ثابت میسازه. بعد از مقداردهی اولیه، دیگه نمیشه تغییرش داد.

• متد final: نمی‌شه در زیرکلاس Override اش کرد.

• کلاس final: نمی‌شه ازش ارث‌بری کرد (مثل String).

! سوال مصاحبه‌ای ۸: رابطه IS-A و HAS-A رو با مثال بگو

• IS-A: وراثت (extends). Dog IS-A Animal.

• HAS-A: ترکیب (Composition). Car HAS-A Engine. (ماشین یه موتور داره، نه اینکه ماشین یه نوع موتور هست). همیشه اولویت با HAS-A است.

! سوال مصاحبه‌ای ۸.۵: چرا Multiple Inheritance در جاوا ممنوعه؟

جواب: به خاطر مشکل Diamond Problem (الماس).

A

/ \

B C

\ /

D

اگه B و C هر دو یه متد از A رو Override کنن، D باید کدوم رو استفاده کنه؟ جاوا با حذف Multiple Inheritance برای کلاس‌ها، این مشکل رو حل کرد. ولی با Interface‌ها این مشکل وجود نداره چون متدهای Interface بدنه ندارن (قبلاً).

راه حل جاوا: یک کلاس می‌تونه چندین Interface پیاده‌سازی کنه.

```
1 interface Flyable { void fly(); }
2 interface Swimmable { void swim(); }
3 class Duck implements Flyable, Swimmable {
4     // می‌تونه هر دو رو پیاده‌سازی کنه بدون مشکل
5 }
```